

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/ Preocupación de diseño

Sincronizar las actualizaciones de información entre los diferentes actores (PM, Colaborador, Administrador, Stakeholder) cuando se realizan operaciones que afectan tareas, tiempos o reportes, evitando inconsistencias y mejorando la trazabilidad en tiempo real.

Orientación de la decisión

Adoptar un patrón Publicador–Suscriptor (Observer) dentro del backend de Django, usando señales y eventos internos para notificar automáticamente a los módulos afectados cada vez que se inserta o actualiza una tarea o registro de tiempo. Complementariamente, se podrán usar WebSockets o notificaciones HTTP para reflejar estos cambios en la interfaz React sin recarga completa.

Supuestos

- Cada actualización en tareas o KPI genera un evento en el backend.
- Los módulos suscritos (por ejemplo, exportes o KPI) reciben la notificación y actualizan sus datos.
- Los usuarios deben ver los cambios reflejados sin refrescar manualmente la interfaz.

Restricciones

- La comunicación entre módulos debe ser asíncrona.
- Se deben limitar los eventos al contexto del usuario afectado.
- No se pueden romper las reglas de permisos del módulo Catálogo.

Requerimientos relacionados

RF05: Notificar automáticamente cambios de estado de tareas o KPIs.

RF06: Actualizar indicadores tras registrar tiempos o comentarios.

RF07: Generar reportes al cierre de una tarea o al alcanzar un umbral KPI.

RNF03: Mantener desempeño bajo múltiples eventos concurrentes.

RNF04: Asegurar la trazabilidad y auditoría de eventos.

Alternativas consideradas

Alternativa 1: Comunicación directa cliente–servidor.

Alternativa 2: Polling periódico.

Alternativa 3: Patrón Publicador–Suscriptor asíncrono.

Evaluación de alternativas

A1:

Ventaja: Simplicidad.

Desventaja Alta dependencia entre módulos.

A2:

Ventaja: Bajo acoplamiento.

Desventaja: Ineficiente por exceso de peticiones.

A3:

Ventaja: Escalable y desacoplado.
Desventaja: Requiere manejo de colas y control de eventos.

**Alternativa
seleccionada y
Justificación**

Se selecciona el patrón Publicador–Suscriptor, ya que permite manejar eventos en tiempo real y mantener sincronización entre módulos y actores sin afectar la estructura modular. Este patrón es coherente con el uso de señales de Django y futuros despliegues con WebSockets o Redis Channels.

**Decisiones
relacionadas**

Implementar señales en Django para los eventos INSERT, UPDATE y DELETE.
Diseñar un componente de notificaciones en React para actualización en tiempo real.

Comentarios

Esta decisión asegura coherencia y visibilidad de cambios entre actores, alineándose con el flujo de eventos descrito en la vista (por ejemplo, “evento de time log” o “evento de actualización”).